

Singular quadrature in `QuadElemList` the self- and near-interaction algorithms, step by step

1 Setting

A `QuadElemList` holds high-order quadrilateral surface patches. Each patch carries an $N \times N$ tensor grid of Gauss–Legendre nodes on the reference square $[0, 1]^2$, and the surface is the order- N polynomial interpolant of those nodes,

$$\mathbf{X}(u, v) = \sum_{i,j} \mathbf{x}_{ij} L_i(u) L_j(v), \quad (u, v) \in [0, 1]^2.$$

A boundary-integral operator needs, for a target $\mathbf{y}$ and a patch E ,

$$U(\mathbf{y}) = \int_E K(\mathbf{y}, \mathbf{x}) \sigma(\mathbf{x}) dS(\mathbf{x}),$$

assembled as a matrix acting on the patch’s nodal density values. When $\mathbf{y}$ is far from E a smooth tensor rule suffices. Two cases need special treatment, and each has its own algorithm:

	self	near
target	<i>on</i> E , at a node (u_0, v_0)	<i>off</i> E , at distance $d \ll$ patch size
integrand	singular, $K \sim 1/r$	nearly singular, sharply peaked
scheme	Duffy edge-collapse	split-at-foot refinement
entry point	<code>SelfInterac</code>	<code>NearInterac</code>

Table 1: Everything else — a target far from the patch relative to its size — takes a plain tensor Gauss–Legendre rule and is not discussed here.

Both entry points map the runtime element order to a compile-time `order` and the runtime tolerance to a runtime `digits`; both are independent and may be called in either order.

2 Self interaction: the Duffy edge-collapse

The target sits *on* the patch, at node (u_0, v_0) , so $r = |\mathbf{X}(u, v) - \mathbf{X}(u_0, v_0)|$ vanishes there and $K \sim 1/r$ is not integrable by any smooth rule. The Duffy transform removes the singularity through the Jacobian rather than by refinement.

Step 1 — split the panel at the target

Join the target to the four corners. This gives four triangles, each with the target as one vertex and one edge of the square opposite it.

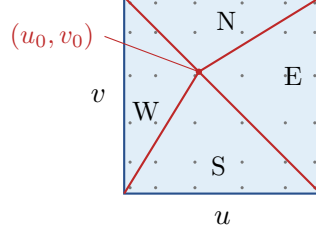


Figure 1: Step 1. The target is always at a node, never at a corner or edge midpoint, so none of the four triangles is degenerate.

Step 2 — collapse one edge onto the target

Parametrise each triangle by $(s, t) \in [0, 1]^2$ with

$$P(s, t) = (u_0, v_0) + s \mathbf{c}(t), \quad \mathbf{c}(t) = (1 - t) \mathbf{a} + t \mathbf{b},$$

where $\mathbf{a}$ and $\mathbf{b}$ point from the target to the two ends of the opposite edge. The line $s = 0$ collapses the entire t -edge onto the target. The Jacobian is

$$|\det| = s |\mathbf{a} \times \mathbf{b}|$$

— linear in s , and *independent of t* .

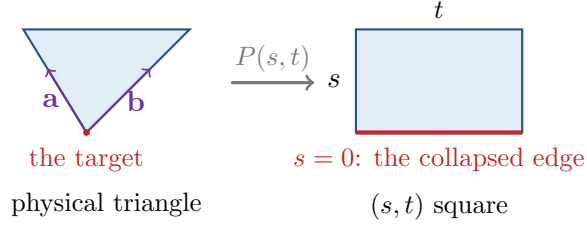


Figure 2: Step 2. The entire edge $s = 0$ maps to the single target point.

That single factor of s is the whole point. For a kernel of order $1/r$, writing $\|\cdot\|_G$ for the length induced by the surface metric G at the target,

$$K |\det| \sim \frac{1}{s \|\mathbf{c}(t)\|_G} \cdot s |\mathbf{a} \times \mathbf{b}| = \frac{|\mathbf{a} \times \mathbf{b}|}{\|\mathbf{c}(t)\|_G},$$

which is bounded as $s \rightarrow 0$. The singularity is gone.

Step 3 — pick the two rules

The s -direction is now smooth, so it takes a plain Gauss–Legendre rule with $q_s = N$ nodes. The t -direction keeps a residual peak: $1/\|\mathbf{c}(t)\|_G$ is largest where $\mathbf{c}(t)$ is shortest, at the foot t^* of the perpendicular from the target to the far edge. The peak has height $\sim 1/d$ and width $\sim d/L$, where d is the distance to that edge and L its length.

Both t^* and d/L are computed in the *surface* metric G , not in parameter space: using parameter distances misplaces the peak by $|\cot \theta|$ peak-widths on a sheared patch. The peak is then resolved by a sinh substitution,

$$t = t^* + \frac{d}{L} \sinh \xi,$$

with one GL rule of n_t points in ξ . n_t grows with the requested digits and is larger for vector kernels than scalar ones.

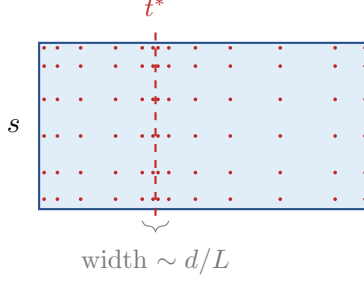


Figure 3: Step 3. One triangle of the four, $n_s \times n_t$ points each.

Step 4 — contract, evaluate, project

Everything above depends only on $(N, t_i, t_j, \text{triangle})$, so the interpolation operators are built once per order and cached. The collapse gives them structure: for the S triangle $v(s, t) = v_0(1 - s)$ depends on s alone, so that factor is a small $(N \times n_s)$ operator rather than $(N \times n_s n_t)$. The evaluation is then a chain of GEMMs:

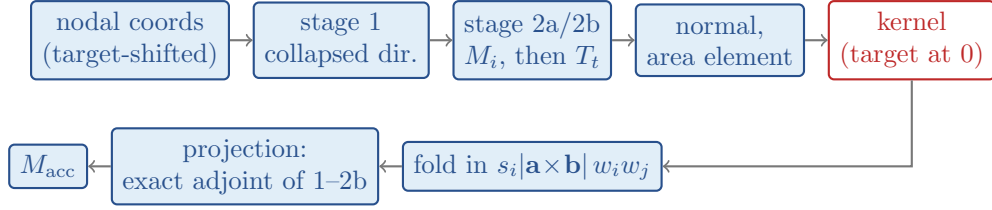


Figure 4: Step 4. One target, one triangle. Blue boxes are GEMMs against operators cached per (N, digits) ; the kernel sees the target at the origin because the nodal coordinates are stored target-shifted.

The projection reuses the same operators in reverse order, so it is the exact adjoint of the interpolation — which is what keeps the assembled operator consistent. Sources are stored relative to the target, so the kernel is always evaluated with the target at the origin and r stays accurate near the singularity.

3 Near interaction: split at the foot

Here the target is *off* the patch at a distance d that may be far smaller than the patch. The integrand is smooth but sharply peaked near the closest point, and a single tensor rule would need impractically many points. The scheme refines — but chooses *where* to refine so that every operator can be precomputed.

Step 1 — find the foot

Locate (u^*, v^*) minimising $|\mathbf{X}(u, v) - \mathbf{y}|^2$ over $[0, 1]^2$: seed at the nearest nodal point, then Gauss–Newton on the first fundamental form, clamped to the box with a backtracking line search, falling back to a shrinking grid search if it stalls. Optimality is tested on the *projected* gradient, since at a box edge the constrained gradient is large — it balances the constraint — and near-pair feet usually do lie on a shared patch edge.

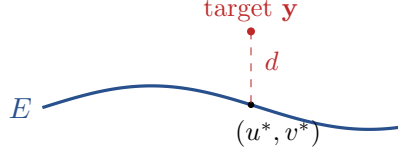


Figure 5: Near, step 1. The foot is found by Gauss–Newton from the nearest nodal point. d may be far smaller than the patch, which is what makes the integrand nearly singular.

Step 2 — split the patch at the foot

Cut the parameter square along $u = u^*$ and $v = v^*$, giving four sub-elements. Every sub-element now has the foot at one *corner*, so refinement always grades toward an *endpoint* of the sub-element rather than to a point in its interior.

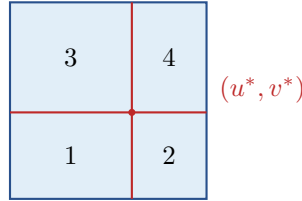


Figure 6: Near, step 2. With the foot at a corner of every sub-element, refinement grades toward an endpoint and the graded intervals depend only on the level — which is what lets the operators be tabulated per N rather than rebuilt per target.

This is the step that makes the scheme cheap. In *normalised* sub-element coordinates the graded intervals depend only on the refinement level, never on (u^*, v^*) , so their interpolation operators are built once per order and looked up. A bisection quadtree, by contrast, leaves the foot mid-cell and needs a position-dependent operator per leaf — hundreds of small matrices rebuilt for every target.

Step 3 — bisect until admissible

Within a sub-element, normalise so the foot is at $x = 1$. The graded intervals are

$$\text{shell}_k = [1 - 2^{-k}, 1 - 2^{-(k+1)}], \quad \text{core}_k = [1 - 2^{-k}, 1],$$

i.e. core_k is the part still touching the foot and shell_k the half turned away from it.

Splitting at the foot leaves the sub-elements *anisotropic*, so quadrissection would hand that aspect ratio down to every descendant. Instead the corner cell is bisected along its longer *physical* dimension only — parameter extent times surface speed — one split at a time, until that dimension is admissible against the target distance:

$$b_{\text{ellipse}} \cdot \max(h_u, h_v) \leq d.$$

Each split emits exactly one leaf (the half not touching the corner), and the u - and v -levels advance independently.

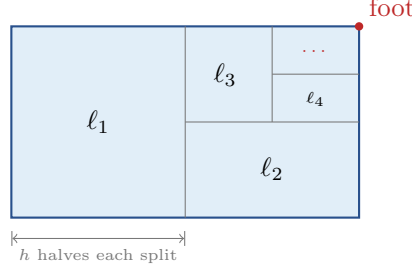


Figure 7: Near, step 3. The corner cell is split along its longer *physical* dimension (parameter extent $\times$ surface speed) until $b_{\text{ellipse}} \max(h_u, h_v) \leq d$, one leaf per split, the u - and v -levels advancing independently. Roughly ten leaves per target at the tolerances used here.

Step 4 — choose the per-cell order

Each leaf gets a Gauss–Legendre rule whose order comes from the tolerance, via a Bernstein-ellipse argument evaluated at the *end-foot* reach (the foot lands on a cell endpoint, which is a weaker requirement than the semi-major reach by $a^2/b^2 \approx 1.9$).

That test lives in parameter space and cannot see one thing: how skewed the patch is where the target sits. The required order is flat up to a corner angle of about 120° and then grows like $1/(180^\circ - \varphi)$ as the corner flattens and the element wraps around the target. So the order is corrected per (element, target) using φ , the acute angle between the surface tangents *at the foot*. An orthogonal parametrisation gives $\varphi = 90^\circ$, a correction factor of 1, and costs a well-shaped mesh nothing.

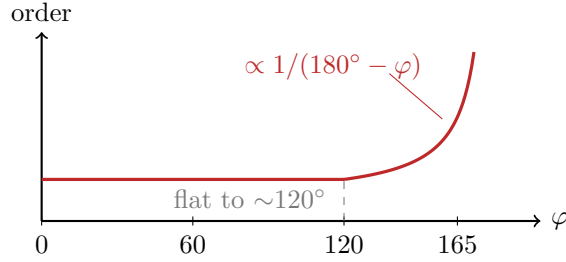


Figure 8: Near, step 4. Schematic, not measured. φ is the acute angle between the surface tangents at the foot; an orthogonal parametrisation sits at $\varphi = 90^\circ$ and needs no correction.

Step 5 — integrate each leaf and accumulate

Each sub-element’s nodal coordinates are built once per target ($X_{\text{sub}} = S_u^\top \mathbf{x} S_v$), so nothing depending on (u^*, v^*) survives into the per-cell loop. Every leaf then runs the same tensor-product block:

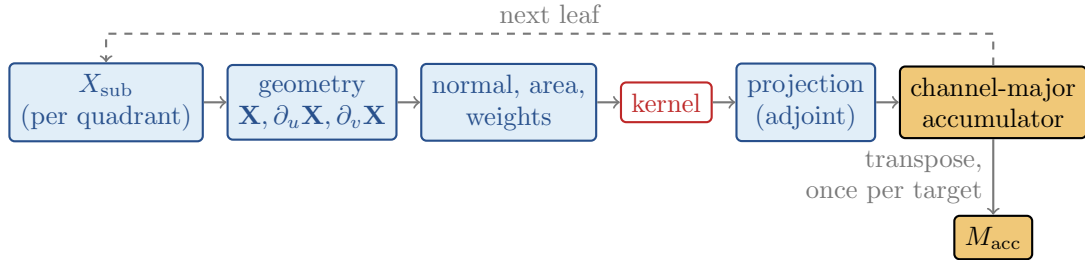


Figure 9: Near, step 5. X_{sub} is built once per target, so nothing depending on (u^*, v^*) enters the loop. The channel-major accumulator lets the projection’s final GEMM add in place ($\beta = 1$).

Cells accumulate into a channel-major buffer so the projection’s last GEMM can add in place ($\beta = 1$); a single transpose into the node-major layout at the end replaces a per-cell sweep.

4 Numerical results

All from `bench-cubed-sphere`. The surface is a cubed sphere twisted about z .

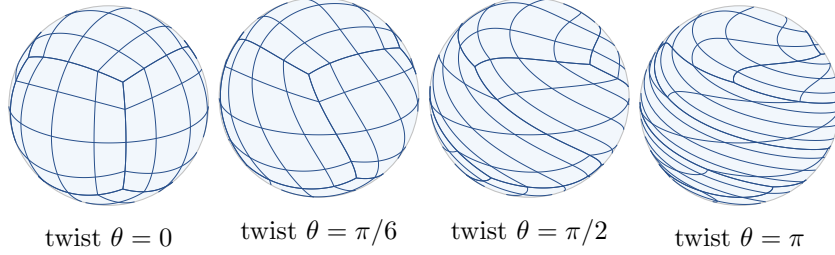


Figure 10: 4×4 patches per face; the runs below use 8×8 and 12×12 . The twist is an *isometry*, so all four panels are the same unit sphere and only the parametrisation shears — every difference in the tables below is therefore caused by the parametrisation alone. At $\theta = \pi$ the patch edges are far from orthogonal, which is where the corner angle φ of Section 3, Step 4 departs from 90° .

4.1 Convergence

Order 12, 12 patches per face (864 elements, 124,416 nodes). *error* is the on-surface Green’s identity $S[\partial_n u] - D[u] = u$, relative to $\max |u|$; *error-density* is the interpolation error of the two densities at off-node parameters, i.e. how well the order-12 patch represents them. The last three columns give the cost of that accuracy on each of the three machines.

kernel, twist	tol	error-density	error	pts/s/core, SL setup		
				Rome	Genoa	Icelake
laplace, $\pi/6$	1.00×10^{-3}	7.18×10^{-14}	2.14×10^{-7}	3628	8547	6677
	1.00×10^{-6}	7.18×10^{-14}	3.73×10^{-10}	3162	6861	6242
	1.00×10^{-9}	7.18×10^{-14}	3.42×10^{-12}	2444	4972	4958
	1.00×10^{-12}	7.18×10^{-14}	3.32×10^{-13}	1631	3247	3591
laplace, $\pi/2$	1.00×10^{-3}	4.89×10^{-11}	2.44×10^{-7}	3326	7287	7739
	1.00×10^{-6}	4.89×10^{-11}	8.52×10^{-10}	2405	4822	4808
	1.00×10^{-9}	4.89×10^{-11}	4.07×10^{-11}	1668	3221	3271
	1.00×10^{-12}	4.89×10^{-11}	2.71×10^{-12}	1066	2104	2149
laplace, π	1.00×10^{-3}	4.28×10^{-8}	2.02×10^{-6}	1944	3822	3867
	1.00×10^{-6}	4.28×10^{-8}	1.63×10^{-8}	943	1839	1701
	1.00×10^{-9}	4.28×10^{-8}	2.04×10^{-10}	539	1031	952
	1.00×10^{-12}	4.28×10^{-8}	4.65×10^{-11}	316	606	551
stokes, $\pi/6$	1.00×10^{-3}	2.25×10^{-13}	4.25×10^{-6}	2277	5024	4681
	1.00×10^{-6}	2.25×10^{-13}	4.66×10^{-9}	1696	3650	3062
	1.00×10^{-9}	2.25×10^{-13}	1.07×10^{-10}	1204	2605	2149
	1.00×10^{-12}	2.25×10^{-13}	7.68×10^{-12}	815	1708	1546
stokes, $\pi/2$	1.00×10^{-3}	1.54×10^{-10}	8.06×10^{-6}	1946	4108	3691
	1.00×10^{-6}	1.54×10^{-10}	3.10×10^{-8}	1219	2139	2199
	1.00×10^{-9}	1.54×10^{-10}	8.89×10^{-10}	800	1693	1443
	1.00×10^{-12}	1.54×10^{-10}	5.98×10^{-11}	521	1093	874
stokes, π	1.00×10^{-3}	1.58×10^{-7}	1.36×10^{-5}	1010	2099	1843
	1.00×10^{-6}	1.58×10^{-7}	1.10×10^{-7}	420	902	782
	1.00×10^{-9}	1.58×10^{-7}	8.76×10^{-10}	235	496	430
	1.00×10^{-12}	1.58×10^{-7}	3.19×10^{-10}	137	286	250

Table 2: Order 12, 12 patches per face (864 elements, 124,416 nodes), single point source outside the surface. *error* is $\max |(S[\partial_n u] - D[u]) - u| / \max |u|$ at the surface nodes, so it exercises both operators together. The two error columns are from Rome; Genoa and Icelake agree to the digits shown. Throughput is the single-layer setup with each machine on all of its cores, so it carries that machine’s parallel efficiency.

- **The geometry is never the limit** — it resolves to $\sim 10^{-15}$ at every twist (not shown).
- **The error tracks the tolerance over nine decades:** at twist $\pi/6$, $2.1 \times 10^{-7} \rightarrow 3.3 \times 10^{-13}$.
- **It stops at a floor set by the parametrisation**, and that floor degrades sharply with twist — at π the best Laplace result is 4.7×10^{-11} against 3.3×10^{-13} at $\pi/6$, a factor of 140 for the same mesh and tolerance.
- ***error-density* is a resolution indicator, not a bound:** at high twist the error sits well below it (4.7×10^{-11} versus 4.3×10^{-8}), because the identity is tested at nodes, where the densities are exact, and the quadrature averages the between-node error.

The throughput columns say where the cost goes, and it is not where the tolerance is.

- **Tolerance is cheap** — nine decades cost about a factor of two (Laplace, $\pi/6$, Rome: 3628 pts/s/core at 10^{-3} , still 1631 at 10^{-12}).
- **Twist is expensive** — at a fixed tolerance of 10^{-9} Laplace falls from 2444 to 539 pts/s/core between $\pi/6$ and π , a factor of 4.5 on Rome and 4.8–5.2 on Genoa and Icelake.

- **So a sheared parametrisation is penalised twice**, once in accuracy and again in time, for the same mesh and the same requested tolerance. Both are the near scheme reacting to the corner angle φ : a skewed patch raises the per-cell order and deepens the refinement, for every target.
- **The machine ratio, by contrast, barely moves** — median 1.98 over all 48 machine-to-Rome comparisons, every one between 1.68 and 2.36. The architecture gap sits in the kernel evaluation, not in how hard the quadrature is working.

4.2 OpenMP scaling

Order 12, 8 patches per face (384 elements, 55,296 nodes), twist $\pi/6$, tol 10^{-9} ; self- and near-interaction setup, with `OMP_PLACES=cores` and `OMP_PROC_BIND=close`. Everything below is the *single-layer* operator alone — the driver times the two setups separately, and the double-layer setup, which takes essentially the same time again, is not included. Three node types:

	cores	AVX-512	build
AMD EPYC 7742 (Rome)	2×64	no	<code>-march=native</code> $\rightarrow$ AVX2
AMD EPYC 9474F (Genoa)	2×48	yes	<code>-march=native</code> $\rightarrow$ AVX-512
Intel Xeon 8358 (Icelake)	2×32	yes	<code>-march=native</code> $\rightarrow$ AVX-512

Table 3: Run `--exclusive`, one MPI rank per node.

machine	kernel	cores	1 thread		full node		speedup
			setup (s)	pts/s	setup (s)	pts/s	
Rome	Laplace	128	15.382	3595	0.183	301 645	$84.1 \times$ (66%)
Genoa	Laplace	96	9.546	5793	0.121	457 565	$78.9 \times$ (82%)
Icelake	Laplace	64	8.764	6310	0.177	313 018	$49.5 \times$ (77%)
Rome	Stokes	128	30.347	1822	0.387	142 835	$78.4 \times$ (61%)
Genoa	Stokes	96	18.117	3052	0.223	248 323	$81.2 \times$ (85%)
Icelake	Stokes	64	18.977	2914	0.410	134 861	$46.3 \times$ (72%)

Table 4: *setup* is the total wall time of the self- and near-interaction setup, single layer only, at order 12, 8 patches per face (384 elements, $N = 55,296$ nodes), twist $\pi/6$, tol 10^{-9} . The double-layer setup costs the same to within a few percent at every thread count on all three machines, so building both operators takes about twice the times listed.

- **Per core at one thread**, Icelake and Genoa are 1.6–1.8 $\times$ Rome (Laplace 6310 and 5793 against 3595). Rome is the only one of the three without AVX-512 and the only one compiled to AVX2, so that is the expected place for the vector width to show.
- **All three lose per-core throughput as threads are added**, Rome most: -34% from 1 thread to 128, against -18% (Genoa, to 96) and -22% (Icelake, to 64).
- **Genoa holds 82–85% efficiency at 96 threads** where Rome is at 61–66% at 128. Part of that is Rome being asked to go further, but Rome is also below Genoa at the same thread count (72% against 89% at 64 threads, Laplace).
- **On absolute throughput the core counts stop cancelling:** Genoa is highest despite having a third fewer cores than Rome, and Icelake matches Rome with half of them.

- **The self loop runs short of work at the widest counts:** only $384/N$ elements per thread, 3 per thread on Rome at 128, which is coarse for `schedule(static)`. That is what bounds every curve at the right-hand end.
- **Stokes sits uniformly a little below Laplace**, consistent with its larger per-target working set.

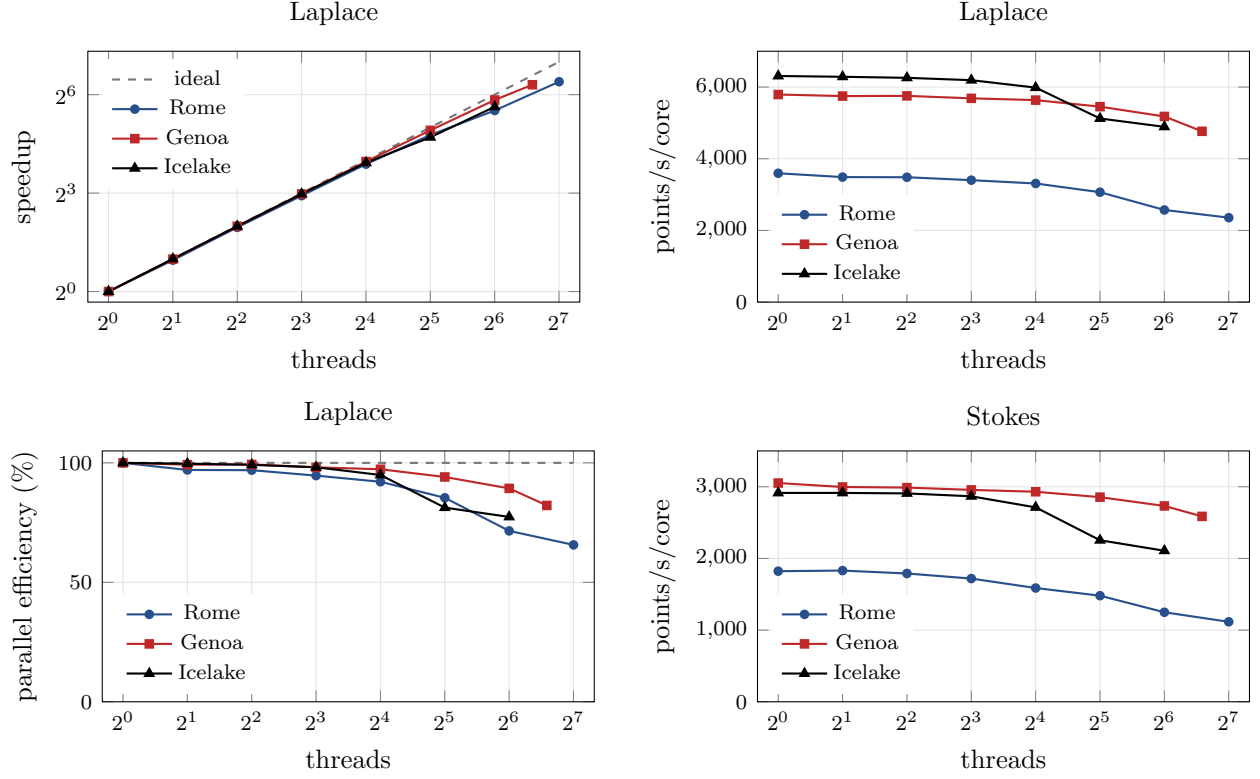


Figure 11: Single-layer setup at order 12, 8 patches per face (384 elements, $N = 55,296$ nodes), twist $\pi/6$, tol 10^{-9} . The double layer is timed separately and is not folded in; it costs about the same again. Speedup and efficiency are against 1 thread on the same machine.